토마토마 React 프론트엔드 초보자 학습 가이드

이 문서는 React를 처음 배우는 초보자를 위해 작성되었습니다. 실제 토마토마 프로젝트 코드를 통해 React의 핵심 개념을 배웁니다.

1. 프로젝트 전체 구조

1.1 폴더 구조 다이어그램

```
tomatoma-frontend
├── src/
│   ├── App.jsx                        (루트 컴포넌트)
│   ├── index.jsx                      (진입점 - React 앱 시작)
│   |
│   ├── pages/
│   |   └── MainPage.jsx                (전체 화면 레이아웃)
│   |
│   ├── components/                    (UI 컴포넌트들)
│   |   ├── Header.jsx                  (상단 헤더)
│   |   ├── Footer.jsx                  (하단 푸터)
│   |   ├── LeftPanel.jsx               (좌측 패널 - 검색/필터/목록)
│   |   ├── SearchInput.jsx             (검색 입력창)
│   |   ├── CategoryFilter.jsx          (카테고리 필터)
│   |   ├── SortDropdown.jsx            (정렬 옵션)
│   |   ├── TrendFoodList.jsx           (트렌드 음식 목록)
│   |   ├── TrendFoodCard.jsx           (음식 카드 - 반복 표시)
│   |   ├── GoogleMapsComponent.jsx     (구글 지도)
│   |   ├── PlaceDetailPopup.jsx        (판매처 상세 정보 팝업)
│   |   └── LoadingSpinner.jsx          (로딩 표시)
│   |
│   ├── hooks/                         (커스텀 훅 - 재사용 로직)
│   |   ├── useTrendFoods.js            (트렌드 음식 데이터 로직)
│   |   ├── useFoodPlaces.js            (판매처 데이터 로직)
│   |   └── useGeolocation.js           (사용자 위치 로직)
│   |
│   ├── services/                      (API 통신 계층)
│   |   ├── api.js                      (Axios 설정)
│   |   ├── trendService.js             (음식 관련 API)
│   |   ├── placeService.js             (판매처 관련 API)
│   |   └── categoryService.js          (카테고리 관련 API)
│   |
│   ├── utils/                         (유틸리티 함수)
│   |   ├── formatUtils.js              (데이터 형식 변환)
│   |   └── mapUtils.js                 (지도 관련 함수)
│   |
│   └── styles/                        (CSS 파일들)
|       ├── App.css                     (전역 스타일)
|       ├── MainPage.css
|       ├── Header.css
|       └── [각 컴포넌트별 CSS...]
|
```

```
├── .env                              (환경변수 파일)
├── vite.config.js                    (Vite 설정)
├── package.json                      (프로젝트 설정, 라이브러리)
└── index.html                        (HTML 진입점)
```

1.2 컴포넌트 트리 구조

사용자가 보는 화면에 컴포넌트들이 어떻게 배치되어 있는지 보여줍니다:

```
App.jsx
└── MainPage.jsx
    ├── Header.jsx
    ├── div.container
    |   ├── LeftPanel.jsx
    |   |   ├── SearchInput.jsx
    |   |   ├── CategoryFilter.jsx
    |   |   ├── SortDropdown.jsx
    |   |   └── TrendFoodList.jsx
    |   |       └── TrendFoodCard.jsx (반복)
    |   |           └── LoadingSpinner.jsx (로딩 중일 때)
    |   |
    |   └── GoogleMapsComponent.jsx
    |       ├── GoogleMap (지도 렌더링)
    |       ├── Marker (마커들)
    |       ├── InfoWindow
    |       |   └── PlaceDetailPopup.jsx
    |       └── LoadingSpinner.jsx (로딩 중일 때)
    |
    └── Footer.jsx
```

1.3 데이터 흐름 (상향식)

```
백엔드 API 서버
     ↓
[api.js] - Axios 설정 및 기본 설정
     ↓
[services/] - trendService, placeService, categoryService
|   └── 실제 API 호출 로직
     ↓
[hooks/] - useTrendFoods, useFoodPlaces, useGeolocation
|   └── 데이터를 받아서 상태(state)로 관리
     ↓
[components/] - React 컴포넌트들
|   └── hooks에서 데이터를 받아 화면에 표시
     ↓
브라우저 화면
```

이 구조를 **"관심사의 분리"**라고 부릅니다. 각 계층이 정해진 역할만 합니다.

---

2. React 핵심 개념

2.1 JSX란 무엇인가?

JSX는 "JavaScript XML"의 줄임말입니다. React에서 HTML처럼 생긴 코드를 JavaScript 안에 직접 쓸 수 있게 해줍니다.

JSX 예시:

```
function TrendFoodCard({ food, isSelected, onSelect }) {
  return (
    <div className={`trend-food-card ${isSelected ? 'selected' : ''}`}>
      <h4 className="food-name">{food.name}</h4>
      <button onClick={onSelect}>상세보기</button>
    </div>
  )
}
```

실제로 변환되는 코드:

```
function TrendFoodCard({ food, isSelected, onSelect }) {
  return React.createElement(
    'div',
    { className: `trend-food-card ${isSelected ? 'selected' : ''}` },
    React.createElement('h4', { className: 'food-name' }, food.name),
    React.createElement('button', { onClick: onSelect }, '상세보기')
  )
}
```

중요한 JSX 규칙:

- HTML 태그는 소문자 시작: `<div>`, `<span>`
- React 컴포넌트는 대문자 시작: `<Header>`, `<TrendFoodCard>`
- 항상 하나의 최상위 요소로 감싸야 함
- `class` 대신 `className` 사용
- `{중괄호}` 안에 JavaScript 변수/표현식 넣을 수 있음

2.2 함수형 컴포넌트 vs 클래스 컴포넌트

토마토마 프로젝트는 **함수형 컴포넌트**만 사용합니다.

함수형 컴포넌트 (현재 사용):

```
function Header({ onRefresh }) {
  return (
    <header className="header">
      <h1>토마토마</h1>
      <button onClick={onRefresh}>새로고침</button>
    </header>
  )
}
```

클래스 컴포넌트 (과거 방식, 요즘은 잘 안 씀):

```
class Header extends React.Component {
  render() {
    return (
      <header className="header">
        <h1>토마토마</h1>
        <button onClick={this.props.onRefresh}>새로고침</button>
      </header>
    )
  }
}
```

함수형 컴포넌트가:

- 코드가 더 간단하고 읽기 쉬움
- Hooks(useState, useEffect 등)를 사용할 수 있음
- 성능이 더 좋음

2.3 Props와 State의 차이

이 두 개념이 헷갈리면 안 됩니다!

Props (프롭스) - "부모에게서 받은 설정값"

- 부모 컴포넌트가 자식 컴포넌트에 전달하는 데이터
- 자식에서는 읽기만 가능, 수정 불가
- 함수의 파라미터처럼 생각하면 됨

```
// 부모 (MainPage.jsx)
<LeftPanel
  selectedFood={selectedFood}          // <- Props
  onSelectFood={setSelectedFood}       // <- Props (함수)
  selectedCategories={selectedCategories}
/>

// 자식 (LeftPanel.jsx)
function LeftPanel({
  selectedFood,            // 받은 Props
  onSelectFood,            // 받은 Props (함수)
  selectedCategories,
}) {
  // selectedFood = "라면" 으로 변경 불가!
  // 대신 onSelectFood("새 음식")으로 부모한테 요청
  return <div>...</div>
}
```

State (상태) - "컴포넌트가 직접 관리하는 변수"

- 컴포넌트가 자체적으로 생성하고 관리하는 데이터
- `useState` 훅으로 생성
- 변경 가능 (setState로만)

```
function MainPage() {
  const [selectedFood, setSelectedFood] = useState(null)
  // selectedFood는 현재 상태
  // setSelectedFood는 상태를 변경하는 함수

  return (
    <LeftPanel
      selectedFood={selectedFood}
      onSelectFood={setSelectedFood}
    />
  )
}
```

정리:

구분	Props	State
소유자	부모	본인
읽기	가능	가능
수정	불가	가능
변경 방법	N/A	setState
비유	함수 파라미터	함수 내부 변수

2.4 useState - 상태 관리 훅

`useState` 는 컴포넌트에서 상태(데이터)를 만들고 관리하는 훅입니다.

기본 문법:

```
const [변수명, 변경함수] = useState(초기값)
```

실제 예시 (MainPage.jsx):

```
function MainPage() {
  const [selectedFood, setSelectedFood] = useState(null)
  const [selectedCategories, setSelectedCategories] = useState(new Set())
  const [searchQuery, setSearchQuery] = useState('')
  const [sortBy, setSortBy] = useState('search_frequency')

  return (
    <div>
      {/* selectedFood는 null 또는 음식 객체 */}
      {selectedFood && <p>{selectedFood.name} 선택됨</p>}

      {/* setSelectedFood로 상태 변경 */}
      <button onClick={() => setSelectedFood({ name: '라면', id: 1 })}>
        라면 선택
```

```
      </button>
    </div>
  )
}
```

동작 흐름:

1. 초기값 설정: `useState(null)` → selectedFood = null
2. 버튼 클릭
3. `setSelectedFood(새로운값)` 호출
4. React가 새 값으로 상태 업데이트
5. 컴포넌트 다시 렌더링 (화면 다시 그리기)
6. JSX의 `{selectedFood?.name}` 에 새 값 표시

주의사항:

```
// 틀린 방법! (절대 금지)
selectedFood = { name: '라면' }  // 직접 수정

// 올바른 방법
setSelectedFood({ name: '라면' })  // setState 함수 사용
```

2.5 useEffect - 사이드 이펙트 관리 훅

`useEffect` 는 컴포넌트가 렌더링될 때 특정 동작을 실행하도록 합니다. 주로 **API 호출**할 때 사용합니다.

기본 문법:

```
useEffect(() => {
  // 실행할 코드
}, [의존성배열])
```

실제 예시 (CategoryFilter.jsx):

```
function CategoryFilter() {
  const [categories, setCategories] = useState([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    // 컴포넌트가 처음 로드될 때 실행됨
    const fetchCategories = async () => {
      try {
        const response = await categoryService.getCategories()
        setCategories(response.data || [])
      } catch (error) {
        console.error('Error:', error)
      } finally {
        setLoading(false)
      }
    }
```

```
    fetchCategories()  // 함수 호출
  }, [])  // 빈 배열 = 처음 로드될 때만 실행

  if (loading) return <div>로딩 중...</div>
  return <div>{categories.map(cat => ...)}</div>
}
```

의존성 배열의 의미:

```
// 1. 의존성 배열이 없음 = 렌더링될 때마다 실행 (위험!)
useEffect(() => {
  console.log('매번 실행됨')
})

// 2. 빈 배열 [] = 처음 로드될 때만 실행
useEffect(() => {
  console.log('처음 1번만 실행됨')
}, [])

// 3. 배열에 변수 포함 = 그 변수가 바뀔 때마다 실행
const [searchQuery, setSearchQuery] = useState('')
useEffect(() => {
  console.log('searchQuery가 바뀔 때마다 실행됨')
  // API 호출 등
}, [searchQuery])
```

TrendFoodList.jsx의 실제 사용 예:

```
function TrendFoodList({
  selectedCategories,
  searchQuery,
  sortBy,
}) {
  const { trends } = useTrendFoods(
    0, 50,
    selectedCategories.size > 0 ? Array.from(selectedCategories)[0] : null,
    sortBy
  )

  // useTrendFoods 내부의 useEffect
  // selectedCategories, sortBy가 바뀌면 자동으로 API 호출
}
```

2.6 useCallback과 useMemo - 최적화 훅

이 두 훅은 **렌더링 성능 최적화**에 사용됩니다. 초보자는 깊이 이해할 필요 없습니다.

useCallback - 함수를 메모리에 저장

```
// GoogleMapsComponent.jsx의 예시
const onLoad = useCallback((mapInstance) => setMap(mapInstance), [])
const onUnmount = useCallback(() => setMap(null), [])

// onLoad, onUnmount 함수를 메모리에 저장해서
// 불필요한 재생성 방지
```

useMemo - 계산 결과를 메모리에 저장

```
// TrendFoodList.jsx의 예시
const filteredList = useMemo(() => {
  let filtered = foodList

  if (selectedCategories.size > 0 && !isSearching) {
    filtered = foodList.filter((food) => selectedCategories.has(food.category))
  }

  if (sortBy === 'search_frequency') {
    filtered = [...filtered].sort((a, b) => (b.searchFrequency || 0) - (a.searchFrequency || 0))
  }

  return filtered
}, [foodList, selectedCategories, sortBy, isSearching])

// 필터링과 정렬 계산 결과를 저장해서
// 의존성이 바뀔 때만 다시 계산
```

2.7 커스텀 훅 - 재사용 가능한 로직

여러 컴포넌트에서 같은 로직을 사용할 때, 훅으로 만들어서 재사용합니다.

useTrendFoods 훅 (hooks/useTrendFoods.js):

```
// 음식 데이터를 가져오고 상태를 관리하는 로직
export const useTrendFoods = (page = 0, size = 20, category = null, sortBy = 'search_frequency') => {
  const [trends, setTrends] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [totalElements, setTotalElements] = useState(0)

  useEffect(() => {
    const fetchTrends = async () => {
      setLoading(true)
      setError(null)

      try {
        const response = await trendService.getTrendingFoods(page, size, category, sortBy)
        if (response.status === 'success') {
          setTrends(response.data.content || [])
          setTotalElements(response.data.totalElements || 0)
```

```
        } else {
          setError(response.message || 'Failed to fetch trends')
        }
      } catch (err) {
        setError(err.message || 'Error fetching trends')
      } finally {
        setLoading(false)
      }
    }

    fetchTrends()
  }, [page, size, category, sortBy])

  return { trends, loading, error, totalElements }
}
```

useTrendFoods 사용 방법 (TrendFoodList.jsx):

```
const { trends, loading, error, totalElements } = useTrendFoods(
  0,           // page
  50,          // size
  null,        // category
  sortBy       // sortBy
)
```

검색 기능을 위한 또 다른 커스텀 훅:

```
export const useSearchTrends = (keyword) => {
  const [results, setResults] = useState([])
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)

  useEffect(() => {
    if (!keyword || keyword.trim() === '') {
      setResults([])
      return
    }

    const search = async () => {
      setLoading(true)
      setError(null)

      try {
        const response = await trendService.searchTrendingFoods(keyword)
        if (response.status === 'success') {
          setResults(response.data.content || [])
        }
      } catch (err) {
        setError(err.message || 'Error searching')
      } finally {
        setLoading(false)
```

```
      }
    }

    const debounceTimer = setTimeout(search, 300)  // 300ms 딜레이
    return () => clearTimeout(debounceTimer)
  }, [keyword])

  return { results, loading, error }
}
```

디바운싱이란? 사용자가 검색창에 "라면"이라고 입력할 때:

- "라" 입력
- "라면" 입력

매번 API를 호출하면 비효율적이므로, **입력이 멈춘 후 300ms가 지나면** API 호출합니다.

다른 커스텀 훅들:

```
// useFoodPlaces.js - 특정 음식의 판매처 가져오기
export const useFoodPlaces = (trendFoodId) => {
  const [places, setPlaces] = useState([])
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null)
  // useEffect로 API 호출
  return { places, loading, error }
}

// useGeolocation.js - 사용자 현재 위치 가져오기
export const useGeolocation = () => {
  const [latitude, setLatitude] = useState(null)
  const [longitude, setLongitude] = useState(null)
  const [error, setError] = useState(null)
  const [loading, setLoading] = useState(true)
  // navigator.geolocation 사용
  return { latitude, longitude, error, loading }
}
```

2.8 Axios + API 호출

Axios는 HTTP 요청을 간단하게 해주는 라이브러리입니다.

api.js - Axios 설정:

```
import axios from 'axios'

const API_BASE_URL = import.meta.env.VITE_REACT_APP_API_BASE_URL || 'http://localhost:8080/api'

const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
```

```
  },
})

export default apiClient
```

설명:

- `axios.create()` : 재사용 가능한 HTTP 클라이언트 생성
- `baseURL` : 모든 요청의 기본 주소 (예: http://localhost:8080/api)
- `headers` : 모든 요청에 포함될 기본 헤더

import.meta.env.VITE_란? Vite 프로젝트에서 환경변수를 읽는 방법입니다.

trendService.js - 실제 API 호출:

```
import apiClient from './api'

export const trendService = {
  getTrendingFoods: async (page = 0, size = 20, category = null, sortBy = 'search_frequency') => {
    try {
      const params = { page, size, sortBy, direction: 'DESC' }
      if (category) params.category = category

      const response = await apiClient.get('/trends', { params })
      // GET http://localhost:8080/api/trends?page=0&size=20&sortBy=search_frequency
      return response.data
    } catch (error) {
      console.error('Error fetching trending foods:', error)
      throw error
    }
  },

  searchTrendingFoods: async (keyword, page = 0, size = 20) => {
    try {
      const response = await apiClient.get('/trends/search', {
        params: { q: keyword, page, size }
      })
      // GET http://localhost:8080/api/trends/search?q=라면&page=0&size=20
      return response.data
    } catch (error) {
      console.error('Error searching trending foods:', error)
      throw error
    }
  },
}
```

async/await란?

```
// 틀린 방법 (콜백 지옥)
trendService.getTrendingFoods()
  .then(response => {
    console.log(response)
```

```
  })
  .catch(error => {
    console.error(error)
  })

// 올바른 방법 (async/await)
async function fetchData() {
  try {
    const response = await trendService.getTrendingFoods()
    console.log(response)  // 응답이 올 때까지 기다림
  } catch (error) {
    console.error(error)  // 에러 발생 시
  }
}
```

---

3. 컴포넌트별 역할 설명

컴포넌트 개요 표

컴포넌트	위치	역할	Props	State
Header	상단	타이틀, 새로고침 버튼	onRefresh(함수)	없음
Footer	하단	마지막 업데이트 시간 표시	없음	lastUpdate
LeftPanel	좌측	검색, 필터, 정렬, 목록 컨테이너	많음(자식 컴포넌트들로 전달)	없음
SearchInput	좌측 상단	검색창	value, onChange	없음
CategoryFilter	좌측 필터	카테고리 선택 버튼들	selectedCategories, onCategoryToggle	categories, loading
SortDropdown	좌측 정렬	정렬 옵션 드롭다운	value, onChange	없음
TrendFoodList	좌측 아래	음식 카드 목록 (useTrendFoods, useSearchTrends)	selectedFood, onSelectFood, 필터정보	없음 (훅에서 관리)

TrendFoodCard	좌측 목록 아이템	각 음식 카드	food, isSelected, onSelect	없음
GoogleMapsComponent	우측	지도 표시	selectedFood, selectedCategories	map, selectedPlace, categories, mapCenter
PlaceDetailPopup	지도 마커 클릭	판매처 상세정보 팝업	place(객체)	없음
LoadingSpinner	여러 곳	로딩 표시	message(선택)	없음

3.1 Header 컴포넌트

파일: `src/components/Header.jsx`

역할:

- 앱 타이틀 "토마토마" 표시
- 새로고침, 도움말, 설정 버튼 제공

코드:

```
function Header({ onRefresh }) {
  return (
    <header className="header">
      <div className="header-content">
        <div className="logo-section">
          <span className="logo">🍅</span>
          <h1>토마토마</h1>
        </div>

        <h2 className="subtitle">트렌드 음식 판매처 찾기</h2>

        <div className="header-actions">
          <button className="btn btn-icon" onClick={onRefresh}>
            ↻  {/* 새로고침 */}
          </button>
          <button className="btn btn-icon" title="도움말">
            ?
```

```
          </button>
          <button className="btn btn-icon" title="설정">
            ⚙️
          </button>
        </div>
      </div>
    </header>
  )
}
```

주요 포인트:

- Props로 `onRefresh` 함수 받음 (부모 MainPage에서 전달)
- 버튼 클릭 시 `onClick={onRefresh}` 호출
- 상태(state) 없음 (단순 표시만)

3.2 SearchInput 컴포넌트

파일: `src/components/SearchInput.jsx`

역할: 검색 입력창

코드:

```
function SearchInput({ value, onChange }) {
  return (
    <div className="search-input-wrapper">
      <input
        type="text"
        className="search-input"
        placeholder="🔍 음식명을 입력하세요..."
        value={value}
        onChange={(e) => onChange(e.target.value)}
      />
    </div>
  )
}
```

동작:

1. 사용자가 입력창에 "라면" 입력
2. `onChange` 이벤트 발생
3. `onChange(e.target.value)` 호출 → 부모의 `setSearchQuery` 실행
4. 부모의 상태 업데이트 → useSearchTrends 실행 → 검색 결과 표시

3.3 CategoryFilter 컴포넌트

파일: `src/components/CategoryFilter.jsx`

역할: 카테고리 선택 버튼들 표시

코드:

```
function CategoryFilter({
  selectedCategories,
  onCategoryToggle,
  onSelectAllCategories,
}) {
  const [categories, setCategories] = useState([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    const fetchCategories = async () => {
      try {
        const response = await categoryService.getCategories()
        if (response.status === 'success') {
          setCategories(response.data || [])
        }
      } catch (error) {
        console.error('Error fetching categories:', error)
      } finally {
        setLoading(false)
      }
    }

    fetchCategories()
  }, [])  // 처음 로드 시만 API 호출

  if (loading) {
    return <div className="category-filter">카테고리 로딩 중...</div>
  }

  return (
    <div className="category-filter">
      <div className="category-buttons">
        {categories.map((category) => (
          <button
            key={category.id}
            className={`category-btn ${selectedCategories.has(category.name) ? 'active' : ''}`}
            style={{
              borderColor: selectedCategories.has(category.name) ? category.color : '#BDBDBD',
              backgroundColor: selectedCategories.has(category.name) ? category.color + '20' :
'transparent',
            }}
            onClick={() => onCategoryToggle(category.name)}
          >
            <span className="category-icon">{category.icon_emoji}</span>
            <span className="category-name">{category.name}</span>
          </button>
        ))}
      </div>

      <div className="category-actions">
        <button className="btn btn-small" onClick={onSelectAllCategories}>
```

```
          {selectedCategories.size > 0 ? '전체 해제' : '전체 선택'}
        </button>
      </div>
    </div>
  )
}
```

주요 포인트:

- `useState` : 백엔드에서 받은 카테고리 목록 저장
- `useEffect` : 처음 로드 시 categoryService.getCategories() 호출
- `.map()` : 배열의 각 카테고리마다 버튼 생성
- `selectedCategories.has()` : Set 자료구조로 선택된 카테고리 확인
- Props로 받은 `onCategoryToggle` 함수 호출로 부모에게 상태 변경 알림

3.4 SortDropdown 컴포넌트

파일: `src/components/SortDropdown.jsx`

역할: 정렬 옵션 선택

코드:

```
function SortDropdown({ value, onChange }) {
  const sortOptions = [
    { value: 'search_frequency', label: '인기순' },
    { value: 'trend_rank', label: '신규순' },
    { value: 'created_at', label: '최신순' },
  ]

  return (
    <select
      className="sort-dropdown"
      value={value}
      onChange={(e) => onChange(e.target.value)}
    >
      {sortOptions.map((option) => (
        <option key={option.value} value={option.value}>
          {option.label}
        </option>
      ))}
    </select>
  )
}
```

동작:

1. 드롭다운에서 "인기순" 선택
2. `onChange` 이벤트 → `onChange('search_frequency')`
3. 부모의 `setSortBy` 실행
4. useTrendFoods의 의존성 배열에 `sortBy` 포함 → API 다시 호출

3.5 TrendFoodCard 컴포넌트

파일: `src/components/TrendFoodCard.jsx`

역할: 하나의 음식 카드 표시

코드:

```
function TrendFoodCard({ food, isSelected, onSelect }) {
  const frequencyBar = formatFrequencyBar(food.searchFrequency, 10000)

  return (
    <div
      className={`trend-food-card ${isSelected ? 'selected' : ''}`}
      onClick={onSelect}
      role="button"
      tabIndex={0}
      onKeyPress={(e) => {
        if (e.key === 'Enter' || e.key === ' ') {
          onSelect()
        }
      }}
    >
      <div className="card-header">
        <div className="category-badge" style={{ backgroundColor: food.color }}>
          {food.category}
        </div>
        <span className="rank">#{food.trendRank}</span>
      </div>

      <h4 className="food-name">{food.name}</h4>

      <div className="frequency-section">
        <div className="label">검색 빈도</div>
        <div className="frequency-bar">
          <div className="bar-fill" style={{ width: frequencyBar.width }}></div>
        </div>
        <span className="frequency-value">{food.searchFrequency?.toLocaleString() || 0}</span>
      </div>

      <div className="card-footer">
        <div className="place-count">판매처: 미로딩</div>
        <button className="btn btn-small" onClick={(e) => e.stopPropagation()}>
          상세보기
        </button>
      </div>
    </div>
  )
}
```

주요 포인트:

- Props: `food` (객체), `isSelected` (boolean), `onSelect` (함수)
- 카드가 선택되면 클래스 이름에 `selected` 추가 (CSS로 하이라이트)
- `formatFrequencyBar()` 함수로 검색 빈도를 막대 길이로 변환
- `onClick={() => onSelect()}` : 카드 클릭 시 선택 상태 변경
- 접근성 개선: `role="button"`, `tabIndex={0}`, `onKeyPress` 추가

3.6 TrendFoodList 컴포넌트

파일: `src/components/TrendFoodList.jsx`

역할: 트렌드 음식 목록 표시, 검색/필터링

코드:

```
function TrendFoodList({
  selectedFood,
  onSelectFood,
  selectedCategories,
  searchQuery,
  sortBy,
  refreshKey,
}) {
  // 트렌드 음식 가져오기
  const { trends, loading: trendsLoading, error: trendsError } = useTrendFoods(
    0,
    50,
    selectedCategories.size > 0 ? Array.from(selectedCategories)[0] : null,
    sortBy
  )

  // 검색 기능
  const { results: searchResults, loading: searchLoading } = useSearchTrends(searchQuery)

  // 어느 데이터를 표시할지 결정
  const isSearching = searchQuery.trim() !== ''
  const foodList = isSearching ? searchResults : trends
  const loading = isSearching ? searchLoading : trendsLoading
  const error = isSearching ? null : trendsError

  // 필터링 및 정렬 (useMemo로 최적화)
  const filteredList = useMemo(() => {
    let filtered = foodList

    if (selectedCategories.size > 0 && !isSearching) {
      filtered = foodList.filter((food) => selectedCategories.has(food.category))
    }

    // 정렬
    if (sortBy === 'search_frequency') {
      filtered = [...filtered].sort((a, b) => (b.searchFrequency || 0) - (a.searchFrequency || 0))
    } else if (sortBy === 'trend_rank') {
      filtered = [...filtered].sort((a, b) => (a.trendRank || 0) - (b.trendRank || 0))
```

```
    }

    return filtered
  }, [foodList, selectedCategories, sortBy, isSearching])

  // 로딩 상태
  if (loading) {
    return (
      <div className="trend-food-list">
        <LoadingSpinner />
      </div>
    )
  }

  // 에러 상태
  if (error) {
    return (
      <div className="trend-food-list error">
        <div className="error-message">
          <p>⚠️ 오류 발생</p>
          <p>{error}</p>
        </div>
      </div>
    )
  }

  // 결과 없음
  if (filteredList.length === 0) {
    return (
      <div className="trend-food-list empty">
        <div className="empty-message">
          <p>🔍 결과가 없습니다</p>
        </div>
      </div>
    )
  }

  // 정상 표시
  return (
    <div className="trend-food-list">
      {filteredList.map((food) => (
        <TrendFoodCard
          key={food.id}
          food={food}
          isSelected={selectedFood?.id === food.id}
          onSelect={() => onSelectFood(food)}
        />
      ))}
    </div>
  )
}
```

핵심 로직:

1. `useTrendFoods` 훅: 전체 트렌드 음식 목록 가져오기
2. `useSearchTrends` 훅: 검색어로 음식 검색
3. 검색 중인지 확인 → 어느 데이터를 표시할지 결정
4. `useMemo` 로 필터링/정렬된 리스트 메모이제이션
5. 로딩/에러/빈 상태/정상 상태 처리
6. `.map()` 으로 각 음식마다 TrendFoodCard 컴포넌트 생성

3.7 GoogleMapsComponent 컴포넌트

파일: `src/components/GoogleMapsComponent.jsx`

역할: 구글 지도 표시, 판매처 마커 표시

주요 기능:

- 사용자 현재 위치 마커 표시 (파란색)
- 선택된 음식의 판매처 마커 표시 (카테고리 색상)
- 마커 클릭 시 상세정보 팝업 표시
- 지도 중심을 사용자 위치로 자동 조정

주요 hooks:

```
const { isLoaded, loadError } = useJsApiLoader({ googleMapsApiKey: apiKey })
const { latitude, longitude } = useGeolocation()
const { places, loading } = useFoodPlaces(selectedFood?.id)
```

3.8 PlaceDetailPopup 컴포넌트

파일: `src/components/PlaceDetailPopup.jsx`

역할: 지도의 InfoWindow에서 판매처 상세정보 표시

정보 표시:

- 가게 이름
- 주소 (📍)
- 전화번호 (☎️) - 클릭 시 전화 걸기
- 웹사이트 (🌐) - 클릭 시 열기
- 평점 (⭐)
- 예상 가격 (💰)
- 영업시간 (🕒)

버튼:

- 전화: `tel:` 링크로 전화 실행
- 웹사이트: `window.open()` 새 탭에서 열기
- 경로안내: 구글 맵 경로 안내 열기

3.9 LoadingSpinner 컴포넌트

파일: `src/components/LoadingSpinner.jsx`

역할: 로딩 중일 때 회전하는 스피너 표시

```
function LoadingSpinner({ message = '로딩 중...' }) {
  return (
    <div className="loading-spinner-wrapper">
      <div className="loading-spinner"></div>
      <p>{message}</p>
    </div>
  )
}
```

---

4. 데이터 흐름 시나리오

시나리오: 사용자가 '한식' 카테고리 클릭

단계별 흐름:

```
1단계: UI 이벤트
┌──────────────────────────────────────────────────────────────┐
│ CategoryFilter.jsx                    │
│ <button onClick={onCategoryToggle}> │
│ 사용자가 '한식' 버튼 클릭             │
└──────────────────────────────────────────────────────────────┘
                  ↓
2단계: Props 함수 호출
┌──────────────────────────────────────────────────────────────┐
│ onCategoryToggle('한식') 실행         │
│ (부모에서 받은 함수)                  │
└──────────────────────────────────────────────────────────────┘
                  ↓
3단계: 부모 상태 변경
┌──────────────────────────────────────────────────────────────┐
│ MainPage.jsx                          │
│ const [selectedCategories,            │
│   setSelectedCategories] = useState() │
│ handleCategoryToggle('한식')          │
│ → Set에 '한식' 추가                   │
└──────────────────────────────────────────────────────────────┘
                  ↓
4단계: TrendFoodList 재렌더링
┌──────────────────────────────────────────────────────────────┐
│ TrendFoodList.jsx                     │
│ useTrendFoods 의존성: [               │
│   ..., selectedCategories, sortBy     │
│ ]                                     │
│ → selectedCategories가 변함           │
│ → useEffect 다시 실행                 │
└──────────────────────────────────────────────────────────────┘
                  ↓
5단계: API 호출
┌──────────────────────────────────────────────────────────────┐
```

```
│ trendService.getTrendingFoods(      │
│   page=0,                           │
│   size=50,                          │
│   category='한식',  ← 변경됨         │
│   sortBy='search_frequency'         │
│ )                                   │
│ → apiClient.get('/trends', {params}) │
│ → HTTP GET 요청 발송                 │
└─────────────────────────────────────┘
                ↓
6단계: 백엔드 응답 처리
┌─────────────────────────────────────┐
│ response.data = {                   │
│   status: 'success',                │
│   data: {                           │
│     content: [                      │
│       { id: 1, name: '라면',         │
│         category: '한식', ... },     │
│       { id: 2, name: '스프',         │
│         category: '한식', ... }      │
│     ],                              │
│     totalElements: 2                │
│   }                                 │
│ }                                   │
└─────────────────────────────────────┘
                ↓
7단계: 상태 업데이트
┌─────────────────────────────────────┐
│ useTrendFoods의 useEffect 내부       │
│ setTrends(response.data.content)    │
│ setLoading(false)                   │
└─────────────────────────────────────┘
                ↓
8단계: 컴포넌트 재렌더링
┌─────────────────────────────────────┐
│ TrendFoodList.jsx                   │
│ trends 상태 변경됨 → 리스트 재생성    │
│ filteredList에서 한식만 필터링       │
│ → TrendFoodCard 컴포넌트들 재렌더링   │
└─────────────────────────────────────┘
                ↓
9단계: 화면 업데이트
┌─────────────────────────────────────┐
│ 사용자 화면                          │
│ 좌측 목록에 한식 음식들만 표시됨     │
└─────────────────────────────────────┘
```

시간순 코드 흐름:

```
// MainPage.jsx
const [selectedCategories, setSelectedCategories] = useState(new Set())
```

```
const handleCategoryToggle = (categoryName) => {
  const newCategories = new Set(selectedCategories)
  newCategories.add('한식')  // ← 여기서 상태 변경
  setSelectedCategories(newCategories)  // ← React에 상태 변경 알림
}

// TrendFoodList.jsx
const { trends, loading } = useTrendFoods(
  0, 50,
  selectedCategories.size > 0 ? Array.from(selectedCategories)[0] : null,
  // selectedCategories가 변함 → '한식' 전달
  sortBy
)

// useTrendFoods.js의 useEffect
useEffect(() => {
  fetchTrends() // ← selectedCategories가 변함 → 자동 실행
}, [page, size, category, sortBy])

// trendService.js
getTrendingFoods: async (page, size, category = '한식', sortBy) => {
  const response = await apiClient.get('/trends', {
    params: { page, size, category, sortBy, direction: 'DESC' }
  })
  // HTTP GET /api/trends?category=한식&...
  return response.data
}
```

---

5. Vite 개발 환경 설명

5.1 npm run dev 할 때 일어나는 일

```
$ npm run dev

                    ↓

vite.config.js 읽기 (프로젝트 설정)

                    ↓

로컬 개발 서버 시작 (localhost:5173)

                    ↓

index.html 로드
  ↓
  <script type="module" src="/src/index.jsx"></script>

                    ↓
```

```
React 애플리케이션 시작
  └── App.jsx
     └── MainPage.jsx
        └── 모든 컴포넌트 렌더링

                  ↓

브라우저 자동 열기 (http://localhost:5173)

                  ↓

파일 저장 감지 → HMR(Hot Module Replacement)
  └── 변경된 부분만 새로고침 (전체 새로고침 불필요!)
  └── 상태 보존
```

5.2 .env 파일 설정

.env 파일 생성:

```
VITE_REACT_APP_API_BASE_URL=http://localhost:8080/api
VITE_REACT_APP_GOOGLE_MAPS_API_KEY=YOUR_GOOGLE_MAPS_API_KEY_HERE
```

파일 위치:

```
tomatoma-frontend/
├─── .env              ← 여기에 환경변수 저장
├─── .env.local        ← 로컬 개발용 (Git에 올리지 말 것)
├─── .env.production   ← 배포용
└─── ...
```

코드에서 사용:

```
// Vite 프로젝트에서는 VITE_ 접두어를 붙여야 함
const API_BASE_URL = import.meta.env.VITE_REACT_APP_API_BASE_URL
// import.meta.env.VITE_REACT_APP_GOOGLE_MAPS_API_KEY

// 빌드 시간에 치환됨 (런타임이 아님)
// 주의: VITE_ 없는 환경변수는 접근 불가
```

5.3 HMR (Hot Module Replacement)이란?

전통적인 웹 개발:

```
1. 코드 수정
2. 브라우저 F5 새로고침
3. 전체 애플리케이션 다시 로드
4. 입력한 데이터 모두 사라짐
```

HMR 사용:

```
1. 코드 수정
2. 파일 자동 저장
```

```
3. 브라우저에서 변경된 부분만 업데이트
4. 입력한 데이터 보존
```

예시:

```jsx
// SearchInput.jsx 수정
function SearchInput({ value, onChange }) {
  return (
    <input
      placeholder="🔍 음식을 검색하세요..."  {/* 수정 */}
      value={value}
      onChange={(e) => onChange(e.target.value)}
    />
  )
}

// 저장하면:
// 1. SearchInput 컴포넌트만 다시 로드
// 2. 부모 컴포넌트는 다시 렌더링하지 않음
// 3. 사용자가 입력한 데이터 유지
// 4. 0.1초 안에 완료
```

5.4 npm run build로 배포판 만들기

```
npm run build

# → dist/ 폴더 생성
# → 모든 JavaScript, CSS 최적화 및 번들링
# → 파일 크기 최소화
# → 배포 서버에 dist/ 폴더만 업로드
```

---

6. CSS 구조 및 스타일링

6.1 CSS 파일 구조

```
src/styles/
├── App.css                 (전역 스타일)
├── MainPage.css
├── Header.css              (Header 컴포넌트 전용)
├── Footer.css
├── LeftPanel.css
├── SearchInput.css
├── CategoryFilter.css
├── SortDropdown.css
├── TrendFoodList.css
├── TrendFoodCard.css
├── GoogleMapsComponent.css
├── PlaceDetailPopup.css
```

```
├── LoadingSpinner.css
└── index.css              (리셋 스타일)
```

6.2 전역 스타일 vs 컴포넌트 스타일

App.css (전역 스타일):

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', sans-serif;
  background-color: #f5f5f5;
  color: #333;
}

.app {
  display: flex;
  flex-direction: column;
  height: 100vh;
}

.container {
  display: flex;
  flex: 1;
  gap: 20px;
}

/* 공통 버튼 스타일 */
.btn {
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
  transition: all 0.2s ease;
}

.btn:hover {
  opacity: 0.8;
}

.btn-small {
  padding: 4px 8px;
  font-size: 12px;
}
```

Header.css (컴포넌트 스타일):

```
.header {
  background-color: #ffffff;
  padding: 20px;
  border-bottom: 1px solid #e0e0e0;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.header-content {
  max-width: 1200px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.logo-section {
  display: flex;
  align-items: center;
  gap: 10px;
}

.logo {
  font-size: 32px;
}

.header-actions {
  display: flex;
  gap: 10px;
}
```

6.3 화이트 테마 구현 방법

화이트 테마의 특징:

- 배경색: 흰색 또는 밝은 회색
- 텍스트색: 어두운 회색 또는 검은색
- 강조색: 한두 가지만 (예: 파란색, 주황색)
- 섀도우: 미묘하고 얕은 그림자

```
/* 색상 변수 정의 */
:root {
  --color-white: #ffffff;
  --color-light-gray: #f5f5f5;
  --color-border: #e0e0e0;
  --color-text-dark: #333333;
  --color-text-light: #999999;
  --color-primary: #2196F3;  /* 파란색 */
  --color-danger: #FF5252;   /* 빨간색 */
}

/* 사용 예 */
```

```
.card {
  background-color: var(--color-white);
  border: 1px solid var(--color-border);
  color: var(--color-text-dark);
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.btn-primary {
  background-color: var(--color-primary);
  color: var(--color-white);
}

.btn-primary:hover {
  background-color: #1976D2;  /* 더 어두운 파란색 */
}
```

---

7. 다음 단계: 구현해야 할 것 (TODO 체크리스트)

완성된 것

- ☑ 기본 레이아웃 (Header, LeftPanel, GoogleMapsComponent, Footer)
- ☑ 검색 기능 (SearchInput + useSearchTrends)
- ☑ 카테고리 필터 (CategoryFilter + useTrendFoods)
- ☑ 정렬 기능 (SortDropdown)
- ☑ 트렌드 음식 목록 (TrendFoodList + TrendFoodCard)
- ☑ 판매처 상세정보 팝업 (PlaceDetailPopup)
- ☑ 로딩 상태 처리 (LoadingSpinner)
- ☑ 사용자 위치 기능 (useGeolocation)
- ☑ 구글 지도 마커 표시 (GoogleMapsComponent)

미완성/개선 필요

- ☐ 다중 카테고리 선택 기능 (현재 첫 번째 카테고리만 사용)

  ```
  // 현재 코드 (TrendFoodList.jsx 16-19줄)
  selectedCategories.size > 0 ? Array.from(selectedCategories)[0] : null
  // ↑ 첫 번째 카테고리만 가져옴

  // 개선 필요: 백엔드가 다중 카테고리 지원하면
  Array.from(selectedCategories).join(',')  // "한식,중식" 형태로
  ```

- ☐ 판매처 개수 표시

  ```
  // TrendFoodCard.jsx 39줄
  <div className="place-count">판매처: 미로딩</div>
  // ↑ "미로딩"으로 고정, useFoodPlaces로 실제 개수 표시 필요
  ```

- ☐ 무한 스크롤 또는 페이지네이션

```
// 현재: size=50으로 고정
// 개선: 스크롤 시 다음 50개 로드
// useCallback으로 onLoad 함수 최적화
```

- [ ] 오류 처리 UI 개선

```
// 현재: 간단한 에러 메시지
// 개선:
// - 재시도 버튼
// - 에러 유형별 다른 메시지
// - 네트워크 상태 표시
```

- [ ] 즐겨찾기 기능 (localStorage 사용)

```
// 예시: 즐겨찾기한 음식 저장
const [favorites, setFavorites] = useState(() => {
  return JSON.parse(localStorage.getItem('favorites') || '[]')
})

const toggleFavorite = (foodId) => {
  const newFavorites = ...
  localStorage.setItem('favorites', JSON.stringify(newFavorites))
}
```

- [ ] 상세 정보 팝업에 더 많은 기능

```
// PlaceDetailPopup.jsx 개선:
// - 리뷰/평점 표시
// - 영업시간 상세 정보
// - 이미지 표시
// - 공유 버튼
```

- [ ] 반응형 디자인 개선

```
/* 모바일 화면에서 좌측 패널 숨기기 */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
  .left-panel {
    display: none;
    /* 또는 오버레이 메뉴로 변경 */
  }
}
```

- [ ] PWA(Progressive Web App) 지원

```
// service worker 등록으로 오프라인 지원
```

- [ ] 성능 최적화

```
// - 이미지 최적화 (lazy loading)
// - 코드 스플리팅
// - 번들 크기 분석
```

- [ ] 타입 안정성 (TypeScript 도입)

```
// props 타입 정의로 버그 조기 발견
interface TrendFoodCardProps {
  food: TrendFood
  isSelected: boolean
  onSelect: () => void
}
```

---

8. 학습 리소스 및 다음 공부 순서

초보자 학습 경로

1단계: React 기초 (1-2주)

- JSX 문법
- 함수형 컴포넌트
- props와 state
- 이벤트 처리

2단계: Hooks (1-2주)

- useState
- useEffect
- useCallback, useMemo
- 커스텀 훅 작성

3단계: API 통신 (1주)

- Axios 기본 사용법
- async/await
- 에러 처리
- 요청/응답 인터셉터

4단계: 실전 프로젝트 (진행 중)

- 토마토마 프로젝트 분석
- 기능 추가 및 개선
- 성능 최적화

추천 학습 자료

공식 문서:

- React 공식 문서: https://react.dev
- Vite 공식 문서: https://vitejs.dev
- Axios 문서: https://axios-http.com

유튜브 채널:

- 생활코딩
- 드림코딩
- 노마드 코더

책:

- "러닝 리액트" (Alex Banks, Eve Porcello)
- "React 완벽 가이드" (Maximilian Schwarzmüller)

---

9. 디버깅 팁

React DevTools 설치

브라우저 확장프로그램 설치:

- Chrome: React Developer Tools
- Firefox: React Developer Tools

디버깅 방법:

```
// 1. console.log 출력
function MyComponent({ data }) {
  console.log('현재 data:', data)
  return <div>{data.name}</div>
}

// 2. 조건부 로깅
useEffect(() => {
  console.log('selectedFood 변경됨:', selectedFood)
}, [selectedFood])

// 3. 브라우저 DevTools 활용
// F12 → Components 탭 → 컴포넌트 선택 → props, state 확인

// 4. Debugger 사용
useEffect(() => {
  debugger  // 여기서 코드 실행 멈춤
  fetchData()
}, [])
```

자주 하는 실수

1. 상태를 직접 수정하기

```
// 틀린 방법
selectedFood.name = '라면'  // 직접 수정 (React가 감지 못함)
```

```
// 올바른 방법
setSelectedFood({ ...selectedFood, name: '라면' })
```

2. useEffect 의존성 배열 잘못 설정

```
// 틀린 방법: 의존성 누락
useEffect(() => {
  fetchData(category)
}, [])  // category가 빠짐 → category 변경해도 안 불림

// 올바른 방법
useEffect(() => {
  fetchData(category)
}, [category])  // category 추가
```

3. setState는 비동기

```
// 틀린 방법
setSelectedFood(food)
console.log(selectedFood)  // 아직 이전 값

// 올바른 방법
setSelectedFood(food)
// useEffect에서 selectedFood 변경 감지
useEffect(() => {
  console.log('변경됨:', selectedFood)
}, [selectedFood])
```

---

10. 자주 묻는 질문 (FAQ)

Q: 왜 컴포넌트를 작게 쪼개나요?

A:

- 재사용성 증대 (TrendFoodCard를 여러 곳에서 사용)
- 테스트 용이
- 유지보수 쉬움
- 성능 최적화 가능

Q: props 드릴링이란?

A: Props를 여러 단계의 컴포넌트를 거쳐 내려주는 것.

```
<MainPage>
  <LeftPanel data={data}>
    <TrendFoodList data={data}>
      <TrendFoodCard data={data} />  // 너무 깊음!
```

해결: Context API 또는 상태 관리 라이브러리(Redux) 사용

Q: 왜 API는 services 폴더에 따로?

A: 관심사의 분리

- components: UI 렌더링만 담당
- hooks: 상태 관리 로직
- services: 백엔드 통신 로직

이렇게 분리하면 나중에 API 엔드포인트 변경해도 컴포넌트는 수정 안 해도 됨.

Q: useCallback과 useMemo는 언제 써요?

A: 성능 문제가 생겼을 때만!

- 예: 큰 배열 필터링이 느려질 때 useMemo
- 예: 자식 컴포넌트가 자주 리렌더링될 때 useCallback

먼저 작성하고 필요하면 나중에 추가.

Q: 타입스크립트 써야 하나요?

A: 초보자는 일반 JavaScript부터 시작. 팀 규모가 커지면 TypeScript 도입.

이 문서로 토마토마 프론트엔드의 기본 구조와 React 핵심 개념을 이해할 수 있기를 바랍니다. 실제 코드를 읽고 수정하면서 배우는 것이 가장 효과적입니다!